Homework 3 (Python version)

ME570 - Prof. Tron

2026-10-20

Goal The goal of this homework assignment is to implement and test the A* algorithm.

Utilities for graphs and priority queues

Question provided 0.1. The following utility complements those given in the previous assignment, and allows you to find the nodes in the graph that are closest to a given point.

> Class name: `Graph`
> File name: `me570_graph.py`
>
> > Method name: `nearest_neighbors`
> > Description: Returns the k nearest neighbors in the graph for a given point.
> > Input arguments
> >
> > - `x_query` (dim. $[2 \times 1]$): coordinates of the point of which we need to find the nearest neighbors.
> > - `k_nearest` (dim. $[1 \times 1]$): number of nearest neighbors to find.
> >
> > Output arguments
> >
> > - `idx_neighbors` (dim. $[$`nb_neighbors` $\times 1]$): indeces in `graph_vector` of the neighbors of `x`. Generally, `nb_neighbors=k`, except when `graph_vector` contains less than k vertices, in which case all vertices are returned.

In this homework, you will mainly use this function with $k = 1$ to find vertices that approximate start and goal locations.

Question provided 0.2. Python provides priority queues via the module `heapq`. With respect to other modules, however, the implementation is based on functions rather than classes. The provided file `me570_pq.py` wraps this implementation in a class, and adds a couple of utility methods that will be useful for the assignment.

> Class name: `PriorityQueue`
> File name: `me570_pq.py`
> Description: A class that implements a priority queue by wrapping functions from the `heapq` module. Each entry consists of a `key` and a `cost`; entries are removed in order of increasing cost.

Method name: `__init__`
Description: Initializes an empty priority queue.

Method name: `push`
Description: Adds a `key` and its associated `cost` to the priority queue.
Input arguments

- `key`: The key identifying the entry.
- `cost`: The priority associated with the entry.

Method name: `is_empty`
Description: Checks whether the priority queue is empty.
Returns arguments

- `flag` (type `bool`): `True` if the queue is empty, and `False` otherwise.

Method name: `is_member`
Description: Checks whether a given key appears in the priority queue.
Input arguments

- `key`: The key to search for.

Returns arguments

- `flag` (type `bool`): `True` if the key appears in the queue, and `False` otherwise.

Method name: `pop`
Description: Removes the entry with the lowest cost, and returns the associated key and cost. Raises an `IndexError` if the queue is empty.
Returns arguments

- `key`: The key identifying the entry.
- `cost`: The priority associated with the entry.

Question Pre-LLM 0.1. Given a list of pairs of costs and keys such as

`groceries=[(2,'Bananas'),(3,'Cantelope'),(1,'Apples')]`,

write a short script that uses the `PriorityQueue` class to print the keys of the list in sorted ascending order. Include your script and the corresponding output in your pre-submission report.

Problem 1: Graph search

In this problem you will implement a graph search algorithm, and apply it to a graph obtained from a grid discretization of a free configuration space.

The graph search function you will develop will be generic, because it can work on a `graph_vector` data structure in a way that is somewhat abstract from the actual problem.

For instance, the function manipulates nodes in terms of their indexes in the data structure, instead of, say, using their coordinates. In this way, the same function can be applied to different problems (e.g., the two-link manipulator).

You will be required to implement the A* algorithm for which the reference pseudo-code is reproduced in Algorithm 1 (this is a essentially the same as the algorithm on page 531 of the book, but includes more details).

Data structures The algorithm uses a priority queue O, and a list of closed edges C. For the priority queue O, you are expected to use the corresponding set of functions from Homework 1. For the list C, you should use a simple array. See Question code 1.5 for further details.

Debugging tips Since A* is a somewhat complex algorithm to implement, you should use the provided function `Graph.plot(_)` and the provided data `graph_test_data.pkl` to test the individual functions and check that the outputs are consistent with what you would expect. In particular, embedding `Graph.plot(_)` and pausing the execution at each iteration

---

Algorithm 1 The A* algorithm. **Ver. 1** is a simplified, complete but not optimal version. **Ver. 2** is the full optimal version.

Add the starting node n_{start} to O, set $g(n_{start}) = 0$, set the `backpointer` of n_{start} to be empty, initialize C to an empty array. ▷ Initialization
repeat
  Pick n_{best} from O such that $f(n_{best}) \leq f(n_O)$ for all $n_O \in O$. ▷ PriorityMinExtract
  Remove n_{best} from O and add it to C.
  Ver. 1 **if** $n_{best} = n_{goal}$ **then**
    Return path using backpointers from n_{goal}. ▷ Path
  Ver. 1 **end if**
  Ver. 2 **if** $g(n_{goal})$ is set **and** $g(n_{goal}) \leq f(n_O)$ for all $n_O \in O$ **then**
    Return path using backpointers from n_{goal}. ▷ Path
  Ver. 2 **end if**
  $S = \text{ExpandList}(n_{best})$
  for all $x \in S$ that are not in C **do** ▷ ExpandElement
    if $x \notin O$ **then**
      Set the backpointer cost $g(x) = g(n_{best}) + c(n_{best}, x)$.
      Set the `backpointer` of x to n_{best}.
      Compute the heuristic $h(x)$. ▷ Heuristic
      Add x to O with value $f(x) = g(x) + h(x)$. ▷ PriorityInsert
    else if $g(n_{best}) + c(n_{best}, x) < g(x)$ **then** ▷ A better path to x has been found
      Update the backpointer cost $g(x) = g(n_{best}) + c(n_{best}, x)$.
      Ver. 2 Update the value of $f(x) = g(x) + h(x)$ in O (might require computing $h(x)$ again). ▷ Heuristic
      Update the `backpointer` of x to n_{best}.
    end if
  end for
until O is empty

---

in the loop of `Graph.search(_)` during debugging is instructive (but remember to remove it in the final version or, even better, use an optional argument to enable it only when needed).

Question code 1.1. Pre-LLM This question must be completed without LLM assistance.

> Class name: `Graph`
> File name: `me570_graph.py`
>
> > Method name: `heuristic`
> > Description: Computes the heuristic `h` given by the Euclidean distance between the nodes with indexes `idx_x` and `idx_goal`.
> > Input arguments
> > - `idx_x` (type `int`), `idx_goal` (type `int`): indexes of the elements in `graph_vector` to use to compute the heuristic.
> >
> > Output arguments
> > - `h_val` (type `float`): the heuristic (Euclidean distance) between the two elements.

Question provided 1.1. The following function demonstrate the use of `Graph.plot`, and visualizes two graphs (already solved with A*) from the provided function `graph_test_data_load(_)`:

> File name: `me570_graph.py`
> Method name: `graph_test_data_plot`
> Description: Visualizes the contents of the file `graph_test_data.pkl` using `Graph.plot(_)` and different sets of visualization options.

In particular, the example `graphVectorMedium_solved` corresponds to the A* example we covered in class. Note that the functions will work properly only after you completed Question code 1.1.

Question Pre-LLM 1.1. Run the provided function `graph_test_data_plot(_)`, and, in your pre-LLM report: *1)* include the two figures with the two solved examples, and *2)* describe the meaning of the arrows, and the numbers labeled with `g`, `h` and `f`.

Question code 1.2.

> Class name: `Graph`
> File name: `me570_graph.py`

Method name: `get_expand_list`

Description: Finds the neighbors of element `idx_n_best` that are not in `idx_closed` (line 11 in Algorithm 1).

Input arguments

- `idx_n_best` (type `int`): the index of the element in `graph_vector` of which we want to find the neighbors.
- `idx_closed` (dim. [`nb_closed` × 1], type `list`): list of indexes containing the list of elements of `graph_vectors` that have been closed (already expanded) during the search.

Output arguments

- `idx_expand` (dim. [`nb_neighborsnotclosed` × 1], type `list`): array of indexes of the neighbors of element `idx_n_best` in `graph_vector`

This correspond to Line 11 of Algorithm 1.

Question code 1.3 (2 points). The function below uses the priority queue from Homework 1.

Class name: `Graph`

File name: `me570_graph.py`

Method name: `expand_element`

Description: This function expands the vertex with index `idx_x` (which is a neighbor of the one with index `idx_n_best`) and updates the attribute `graph_vector` and returns an updated version of `pq_open`.

Input arguments

- `idx_n_best` (type `int`), `idx_x` (type `int`), `idx_goal` (type `int`): indexes in `graph_vector` of the vertex that has been popped from the queue, its neighbor under consideration, and the goal location.
- `pq_open` (type `PriorityQueue`): object of type `PriorityQueue` with the priority queue of the open nodes.

Output arguments

- `pq_open` (type `PriorityQueue`): Same as the homonymous input argument, but updated with the new nodes that have been opened.

This function corresponds to Lines 13–22 in Algorithm 1.

Question code 1.4. Implement a function that transforms the backpointers describing a path into the actual sequence of coordinates.

Class name: `Graph`

File name: `me570_graph.py`

Method name: `path`
Description: This function follows the backpointers from the node with index `idx_goal` in `graph_vector` to the one with index `idx_start` node, and returns the *coordinates* (not indexes) of the sequence of traversed elements.
Input arguments

- `idx_start` (type `int`), `idx_goal` (type `int`): indexes in `graph_vector` of the starting and end vertices.

Output arguments

- `x_path` (dim. [2 × `nb_path`]): array where each column contains the coordinates of the points obtained with the traversal of the backpointers (in reverse order). Note that, by definition, we should have `x_path[:,0]` equal to `graph_vector[idx_start]['x']` and `x_path[:,-1]` equal to `graph_vector[idx_goal]['x']`.

Question code 1.5. This question puts together the answers to Questions code 1.1–code 1.4.

Class name: `Graph`
File name: `me570_graph.py`

Method name: `search`
Description: Implements the A* algorithm, as described by the pseudo-code in Algorithm 1.
Input arguments

- `idx_start` (dim. [1 × 1]), `idx_goal` (dim. [1 × 1]): indexes in `graph_vector` of the starting and end vertices.

Output arguments

- `x_path` (dim. [2 × `nb_path`]): array where each column contains the coordinates of the points of the path found from `idx_start` to `idx_goal`.

Requirements: **optional** The function will need to initialize the fields `g` and `backpointer` to be, respectively, an empty list and the `None` value for each node in the graph, with the exception that `g` = 0 for the `idx_goal` node. Set a maximum limit of iterations in the main A* loop on line 2 of Algorithm 1. This will prevent the algorithm from remaining stuck on malformed graphs (e.g., graphs containing a node as a neighbor of itself), or if you make some mistake during development.

This function corresponds to the entire Algorithm 1; you can choose to implement either the lines corresponding to **Ver. 1** (slightly simpler but not guaranteed to produce optimal paths) or to **Ver. 2** (which guarantee optimal paths); the unlabeled lines in black are common to both versions. For the purposes of this homework, you can assume that a path always exists (although this can be optionally generalized in Question optional 1.1).

The function for the cost to use in the priority queue, denoted as $f(n)$ in the book and in Algorithm 1, is $f(n) = g(n) + h(n)$, where $g(n)$ is the cost of the path from node n to the start vertex (going through the backpointer), and $h(n)$ is the heuristic (the Euclidean distance between nodes). The cost $c(n, x)$ between two vertices is the one stored in the `neighbors_cost` field.

Your implementation of `Graph.search(_)` must contain the following elements:

- a priority queue `pq_open` of the *open* vertices (the structure O in Algorithm 1); this structure must be an object instantiated from the class `PriorityQueue` from Homework 1; use the index of the vertex for the *key* and the function $f(n)$ described above for the `cost`.
- a list `idx_closed` (dim. [`nb_closed` × 1]), corresponding to the set C in Algorithm 1 containing the indexes of the closed vertices.

Question optional 1.1. Add conditions to return an empty `path` if A* cannot find a feasible path.

Question optional 1.2. Add an argument `method` containing a string that determines the behavior of the algorithm. The function $f(n)$ will then depend on the value of the argument `method`:

- $f(n) = g(n)$ if `method` is equal to `bfs`.
- $f(n) = h(n)$ if `method` is equal to `greedy`.
- $f(n) = g(n) + h(n)$ if `method` is equal to `astar`.

Question report 1.1. Make a function `graph_search_test(_)` that test your graph search function on one of the provided graphs.

> File name: `me570_hw4.py`
> Method name: `graph_search_test`
> Description: Call `graph_search(_)` to find a path between the bottom left node and the top right node of the `graphVectorMedium` graph from the `graph_test_data_load(_)` function (see Question ??). Then use `Graph.plot(_)` to visualize the result.

Include the figure in your report, and visually check the result with the solution given by `graphVectorMedium_solved` (see the previous Homework assignment).

This will give you the confidence that your A* algorithm works well before moving on. **optional** Add an optional argument to `Graph.search(_)` to enable/disable plots of the solution after every node expansion, so that you can verify each step of the algorithm.

Problem 2: Application of A* to the two-link manipulator

In this problem you will apply the graph search function you implemented in Problem 1 to the two-link manipulator from Homework 2. In this case, the coordinates in the field `'x'` of `graph_vector` will represent the pairs of angles (θ_0, θ_1) for the two links (as was specified in Homework 2).

The file `twolink_graph_free_space.pkl` contains an object of type `Graph` that describes the free space of the two-link manipulator with respect to the obstacle points contained in `twolink_testData.mat`; this is essentially the result of Question report 3.1 from Homework 2 (please reread that question for details).

Question provided 2.1. This method loads the data from the file `twolink_graph_free_space.pkl`

> File name: `me570_robot.py`
> Class name: `TwoLinkGraph`
> Description: A class for finding a path for the two-link manipulator among given obstacle points using a grid discretization and A*.
>
> > File name: `me570_robot.py`
> > Method name: `load_free_space_graph`
> > Description: Loads the contents of the file `twolink_graph_free_space.pkl`, and stores it in the `TwoLinkGraph.graph` attribute of the class.

Question Pre-LLM 2.1. Note that the graph returned by `load_free_space_graph()` does not consider the fact that nodes with the same values of θ_0 (respectively, θ_1) and θ_1 (respectively, θ_0) equal to 0 or 2π should be considered equivalent. Explain what potential problems this might cause, and solutions to fix it. The questions below offer some clues.

Question optional 2.1. Modify the `TwoLinkGraph` class to work with the topology of the configuration space of the two-link manipulator by following the steps below:

1) Add an optional `mode='torus'` option to the method `load_free_space_graph()` which connects all the nodes on the "left" edge ($\theta_0 = 0$) to the corresponding nodes on the "right" edge ($\theta_0 = 2\pi$), and all nodes on the "top" edge ($\theta_1 = 0$) to the corresponding nodes on the "right" edge ($\theta_1 = 2\pi$) with edges that have zero cost.

2) Modify `Graph.heuristic()` to allow an additional optional argument `mode='torus'`. With this argument, the heuristic will use mod-2π arithmetic to compute the distance between pairs of angles instead of the Euclidean distance (look at the function `Edge.angle()` from Homework 1 for inspiration). For instance, with this option the heuristic between the pairs of angles $\begin{bmatrix} 2\pi - 0.1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0.1 \\ 0 \end{bmatrix}$ should be 0.2.

3) Modify `Graph.search()` with an optional argument that enables the use of `graph_heuristic` with the `'torus'` option (you can either introduce an option `'torus'`, or allow passing the heuristic as a function).

4) Modify the method `TwoLinkGraph.search_start_goal()` that uses the modified `Graph.search()` from the previous point.

Question code 2.1. Use the method `Graph.search()` to find a collision-free path for the two-link manipulator.

Class name: `TwoLinkGraph`
File name: `me570_robot.py`

Method name: `search_start_goal`
Description: Use the method `Graph.search(_)` to search a path in the graph stored in `graph`.
Input arguments

- `theta_start` (dim. [2 × 1]), `theta_goal` (dim. [2 × 1]): vectors describing the initial and final joint angles for the path search.

Output arguments

- `theta_path` (dim. [2 × `nb_path`]): a sequence of pairs of angles describing a feasible path.

Question **Use LLM** **2.1.** Use an LLM to create the following method.

Class name: `TwoLink`
File name: `me570_robot.py`

Method name: `plot_animate`
Description: Displays the configurations along `theta_path` as an animation, using `TwoLink.plot(_)` to draw each configuration. Pauses for $\frac{1}{\texttt{fps}}$ seconds between draw operations. The drawings accumulate on the same figure, so the configurations overlap and remain visible after the animation.
Input arguments

- `theta_path` (dim. [2 × `NPoses`]): A sequence of configurations, with each column specifying one configuration along the path.
- `fps`: The animation frame rate in frames per second.
- `decimate` (default: `1`): **optional** A positive integer specifying how many poses to skip between displayed configurations. The function displays poses with zero-based indices 0, `decimate`, 2`decimate`, etc.

Question **report** **2.1.** **quiz** Plot the points `obstacle_points` in `twolink_testData.mat` (see the code that was provided under Question report 2.2 in Homework 2), call the function `TwoLinkGraph.search_start_goal(_)`, and then the method `TwoLink.plotAnimate(_)`, for the following start/goal configurations:

- *Easy*: `theta_start`$=\begin{bmatrix}0.76\\0.12\end{bmatrix}$, `theta_goal`$=\begin{bmatrix}0.76\\6.00\end{bmatrix}$.
- *Medium*: `theta_start`$=\begin{bmatrix}0.76\\0.12\end{bmatrix}$, `theta_goal`$=\begin{bmatrix}2.72\\5.45\end{bmatrix}$.

- **optional** *Hard*: `theta_start=`$\begin{bmatrix}3.30\\2.34\end{bmatrix}$, `theta_goal=`$\begin{bmatrix}5.49\\1.07\end{bmatrix}$. For this case, the planner will find a feasible path only if you implement and pass the `'torus'` option.

Note that all values for the angles are in radians. Every time the graph search finds a feasible path, you should see the manipulator move between the obstacle points, where each configuration that is plotted is not in collision. For each path, include two plots in your report

1) The plots produced by `TwoLink.plot_animate(_)` in your report.

2) A plot of the graph with `TwoLinkGraph.graph.plot(_)`, superimposing the plot of `theta_path` to see the path found by `A*` in the graph.

optional Use the function `time(_)` from the module `time` to measure the time required by the graph search function in each case.

Question report 2.2. For the *Easy* case in the question above, comment on the *unwinding* phenomenon that appears if you do not use the `'torus'` option (that is, why the planner does not find the straightforward path that keeps the first link fixed). To obtain full marks, make sure to include a figure (similar to the ones produces by Question report 2.1) showing the two alternative paths (the unwinding one, and the direct one); if you have not implemented the `'torus'` option, you can hand-draw an approximate expected solution. .

Question report 2.3. Comment on how close the planner goes to the obstacles, and what you could do about it in a practical situation.

Question optional 2.2. Notice that the majority of the time during planning is spent in checking collisions while generating the free space graph, but most of the graph is never actually explored during search. To significantly speed up the planner, you can use *lazy evaluation*. Lazy evaluation performs collision checking when looking for neighbors in the expansion of a node (line 11 in Algorithm 1), instead of performing it for all the nodes at the beginning. Make a method `TwoLinkGraph.search(_)` that is the same as `Graph.search(_)` but:

- The attribute `graph_vector` does not contain neighbor information (the fields `neighbors` and `neighbors_cost` are not used).
- The subfunction `getExpandList(_)` uses `TwoLink.is_collision(_)` to find the neighbors of the node being expanded.

Run the function `TwoLinkGraph.search(_)` on the problems above, and compare the computation times with the previous implementation.

> "The last function of reason is to recognize that there are an infinity of things which surpass it."
> — *Blaise Pascal via goodreads.com*